AfriCoT-VLA: Cloud-Hosted Chain-of-Thought Reasoning with an Edge Latent World Model and Fallback for Intermittent Connectivity in Sub-Saharan African Traffic Signal Control

Temitope W. Ariyo[1*] and Ebun P. Fasina[1]

[1]Machine Learning Research Group, Department of Artificial Intelligence and Robotics, Faculty of Computing and Informatics, University of Lagos, Akoka, Lagos, Nigeria

temitopeariyo6@gmail.com, efasina@unilag.edu.ng

**Corresponding author*

Abstract

Traffic signal control in Sub-Saharan African cities has to cope with mixed informal transit, intermittent connectivity and power, sparse sensing and a need for decisions that regulators can audit. This paper proposes AfriCoT-VLA, a cloud-hosted design that follows a cloud-based eight-layer framework set by the second author. In the intended deployment, the data store, the vision-language-action model with chain-of-thought reasoning, and the explainability, service, agent and autonomous layers are hosted in the cloud, and the network layer carries compressed keyframes, not continuous video, to the slow loop, which turns a keyframe and its context into a compact subgoal with a written justification. An on-site latent world model and cross-entropy method planner, the fast loop, choose phase timings against that subgoal, and together with a rule-based safety shield they form a resilience tier that keeps the junction operating when the link fails. A capability-aware coordinator governs degradation and every decision is logged for audit. No cloud deployment was built or measured. A SUMO prototype of one T-junction, run over five seeds in four scenarios, examined the control logic. On held-out data the world model lowered one-step queue error against a persistence guess by 69.4 percent. The fast loop with a default subgoal was behind max-pressure control in the means on normal and incident demand and comparable on market demand, and the variant without the world model was no better than fixed-time control. The shield recorded 0 safety violations in 100 evaluation runs. Negative results are reported in full. The full system with the vision-language slow loop is implemented and wiring-tested but not yet evaluated. All findings come from one simulated junction and are exploratory.

Keywords

traffic signal control; chain-of-thought reasoning; vision-language-action models; latent world models; explainability; Sub-Saharan Africa

1 Introduction

1.1 Background and motivation

Cities in Sub-Saharan Africa (SSA) are beginning to equip junctions with cameras and software-controlled signals. Lagos has been reported to have selected a vendor for an intelligent traffic system (Olomu, 2025), the Lagos Metropolitan Area Transport Authority has been reported to be deploying AI-supported systems on selected roads (ITEdgeNews,

2026), and Nairobi has announced camera-equipped, centrally coordinated signals in phases (Kenya Urban Roads Authority, 2023; Kinyanjui, 2024; Mumbi, 2026). The conditions under which such controllers must run are demanding (Adeyemo, 2026). Hundreds of millions of people in the region still lack electricity (International Energy Agency et al., 2025), mobile internet adoption remains well below the world average (GSMA, 2024a, 2024b), and the compute available at a roadside cabinet is small. A junction controller for these settings has to treat power, bandwidth and compute as design inputs. The deployment studied here is intended to be hosted in the cloud, and because a cloud-only controller would be fragile under these constraints, the design keeps a small resilience tier at the junction.

The traffic itself is also different from the conditions assumed by most signal-control research. Motorcycle taxis (okadas), tricycles (kekes) and minibuses (danfos, matatus) share the carriageway with cars, motorcycles filter between lanes, and pedestrians cross at the same junctions in large numbers on market days. Demand changes with the calendar and with incidents, and a controller tuned for a typical peak may serve a market day poorly. Finally, public authorities such as the Lagos State Traffic Management Authority and the Ministry of Transportation are expected to be able to ask why a signal behaved as it did. A controller that cannot give a reviewable account of its decisions is difficult to adopt, however well it performs.

Two families of models are attractive for this problem for complementary reasons. Vision-language-action (VLA) models extend pretrained vision-language models so that images and text can be mapped to actions (Kim et al., 2024; Zitkovich et al., 2023), and several have been trained to write out intermediate reasoning before they act (Yang et al., 2025; Zawalski et al., 2024). Such models can read context that numerical sensors do not capture, and the reasoning is text that a person can read. Latent world models, on the other hand, learn a compact predictive model of an environment and let a planner imagine the consequences of candidate actions cheaply (Hafner et al., 2019; Hafner et al., 2025; Hansen et al., 2024). A large reasoning model is too slow and too connectivity-dependent to set signal timings every few seconds, whereas a small world model is cheap enough to do so on site but has no access to semantic context. The design studied here couples the two, with the large reasoning model hosted in the cloud and the small world model at the junction.

1.2 Relation to the authors' earlier work

This paper is an offshoot of a position paper by the same authors (Ariyo & Fasina, 2026), which argued that a slow, infrequent subgoal generator and a fast, on-device world-model planner should be linked through a narrow interface, and which compared candidate interfaces using a toy two-dimensional study. That paper contained no traffic domain, no chain-of-thought reasoning, no safety layer and no simulation of a real control problem. The present paper keeps the two-timescale idea and the cross-entropy planner, places the slow loop and the upper layers in a cloud-hosted deployment with an on-site resilience tier, and adds the following.

1. A slow loop based on chain-of-thought reasoning that produces a subgoal together with a written justification, and a log that records both for audit.

2. A rule-based safety shield and an independent safety audit, which together define what the learned components may and may not do.
3. A capability-aware coordinator and explicit degradation tiers, including the behaviour when the slow loop is unreachable.
4. A description of the intended cloud deployment architecture, which is specified and not built, and a SUMO prototype of one Lagos-style T-junction, with results for the parts that have been evaluated and an explicit account of the results that are negative.

The paper reports no cloud measurement of any kind, and it does not claim that the full system improves traffic control: the slow loop is implemented and wiring-tested, but its evaluation with a real vision-language model has not yet been run.

1.3 Research questions and contributions

Three questions guide the work. First, can a small latent world model trained on simulated perception output predict short-horizon queue evolution well enough to support planning? Second, how does a world-model planner with a fixed default subgoal compare with fixed-time, actuated and max-pressure control at one junction under four demand conditions, and which of its parts matter? Third, can the safety, audit and degradation machinery around the learned components be specified so that it can be tested independently of them? The contributions are the architecture and its interfaces, and the prototype with the evidence for the parts that were evaluated, including what failed.

2 Related Work

2.1 Traffic signal control

A survey by Wei et al. (2019c) organises the fixed-time, actuated, adaptive and learning-based approaches to signal control. Max-pressure control selects the phase that relieves the largest pressure between upstream and downstream queues, and was shown to be throughput-optimal under stated assumptions for a network of signalised intersections (Varaiya, 2013). It needs no model of demand, which makes it a demanding baseline and the one against which the present prototype is mainly compared. Reinforcement-learning controllers include PressLight, which learns a max-pressure-inspired reward for arterial coordination (Wei et al., 2019a), CoLight, which uses graph attention for network-level cooperation (Wei et al., 2019b), and multi-agent methods for large networks (Chu et al., 2020). They typically assume dependable sensing and communication. LLMLight casts a large language model as a signal-control agent that reads textual descriptions of traffic (Lai et al., 2023). It is the closest existing work to the slow loop in motivation, but it places the language model directly in the control path, whereas the present design lets it set priorities only.

2.2 Latent world models and sampling-based planning

Planning in a learned latent space was popularised by PlaNet, which used the cross-entropy method to search over action sequences (Hafner et al., 2019). The cross-entropy method repeatedly refits a sampling distribution to the best-scoring candidates (de Boer et al., 2005; Rubinstein, 1999), and sampling-based planning with learned dynamics has been used in

sample-efficient model-based reinforcement learning (Chua et al., 2018). Later systems trained policies in imagination (Hafner et al., 2025) or combined learned latent models with model predictive control (Hansen et al., 2024), and joint-embedding predictive models have been scaled to video (Assran et al., 2025). The world model used here is a small recurrent latent model in their general style, not an implementation of any of them, trained on noisy per-approach summaries and not on pixels.

2.3 Vision-language-action models and chain-of-thought reasoning

OpenVLA, Octo and π0 are generalist policies for manipulation (Black et al., 2025; Kim et al., 2024; Octo Model Team et al., 2024), RT-2 showed that web knowledge can transfer to robot control (Zitkovich et al., 2023), and a survey covers the area (Sapkota et al., 2025). Embodied chain-of-thought training teaches a policy to write intermediate reasoning steps before it acts (Zawalski et al., 2024), and InstructVLA combines multimodal reasoning with action generation through instruction tuning (Yang et al., 2025). These works address manipulation, where the action is a low-level robot command. In the present design the model's action is a subgoal, a small vector of cost weights, and the reasoning text is stored as the justification of the decision.

2.4 Hierarchical and dual-system designs

Splitting a slow semantic component from a fast control component is an established pattern. Hi Robot has a vision-language model issue language commands to a lower-level policy (Shi et al., 2025), GR00T N1 and Helix pair a slower vision-language module with a faster action module (Figure AI, 2025; NVIDIA et al., 2025), and Director learns goals in a latent space for a worker policy (Hafner et al., 2022). The authors' earlier paper compared several of these in detail (Ariyo & Fasina, 2026). What distinguishes the present design is the combination of a narrow cost-weight interface, an explicit planner with a learned model in the fast loop, a rule-based shield that no learned part can bypass, and degradation rules written for unreliable connectivity.

2.5 Safety, explainability and constrained deployment

Benchmarks and frameworks for the safety of embodied agents that are driven by language models have started to appear (Yin et al., 2024; Zhang et al., 2024), and they motivate keeping learned components away from the final actuation decision. Liu et al. (2026) argue for an infrastructure-first approach to embodied AI in the Global South, and foundation models carry risks that systems built on them inherit (Bommasani et al., 2021; Feng et al., 2025). None of this literature gives a signal-control design in which the infrastructure limits, the audit trail and the safety rules are first-order inputs, which is the gap the present paper addresses.

3 System Design

The design is described in full, including the cloud deployment and other parts that the prototype does not implement. A table in the experimental section lists what was built and what is only specified.

3.1 Two-timescale architecture

At each junction, an edge computer is connected to the existing signal controller. Edge perception converts camera input into per-approach queue length, counts by vehicle class, occupancy and the number of waiting pedestrians at each crossing. Raw video is intended to stay on site. The fast loop runs on the same edge computer at every signal decision, which in the prototype is about 199 decisions per simulated run, and it is never routed elsewhere. The slow loop runs every few minutes, or earlier on unusual events, and it is intended to run in the cloud when connectivity allows, and otherwise on a cluster server or on the edge computer.

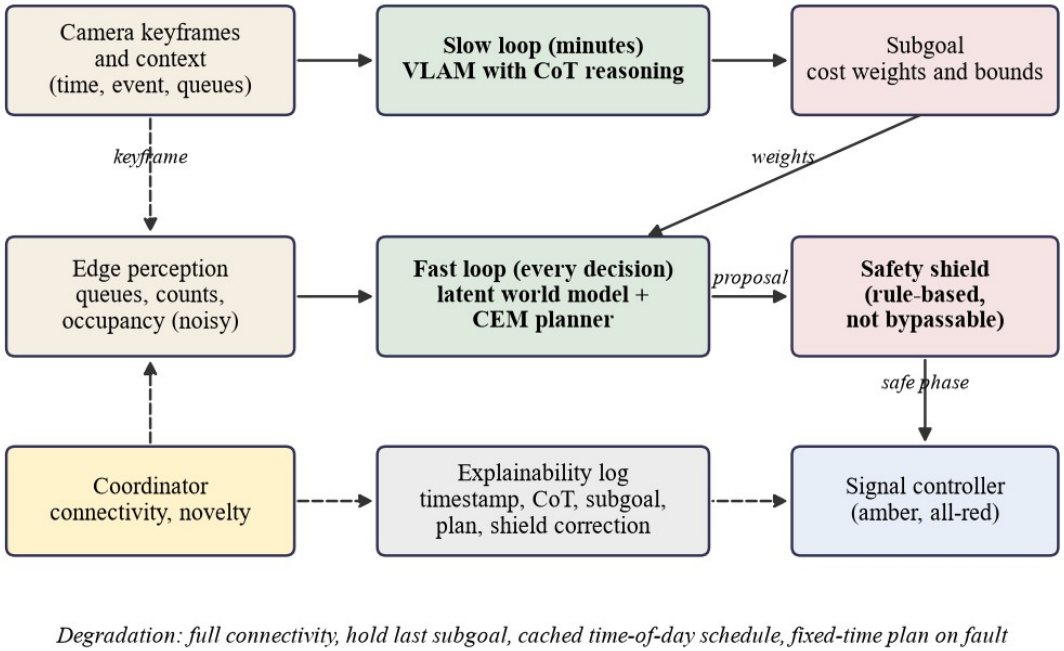


Figure 1**.** The two-timescale architecture. Solid arrows show the main data flow, dashed arrows show keyframes, coordination and logging. The subgoal is the only interface between the slow and fast loops, and the shield stands between the planner and the signal controller

The figure above summarises the data flow. The slow loop receives a keyframe and a context block (clock time, known events such as a market day or a reported incident, the current queues and the current subgoal). A vision-language-action model (VLAM) is asked to reason step by step and then to emit a subgoal. The fast loop receives the subgoal as the weights of its cost function. A latent world model predicts the consequences of candidate signal plans, a cross-entropy method (CEM) planner searches for the plan with the lowest cost, and only the first step of the best plan is passed to the shield. The VLAM and the world model are deliberately kept as separate models that are linked only by the subgoal. Unifying them in one model is an open research problem, and coupling is a design choice made under the compute and connectivity limits described above, not a claim that a unified model would be inferior.

3.1.1 Fast loop: latent world model and CEM planner

The observation vector contains the noisy perception output, the current phase, the time in phase and the normalised time of day. An encoder maps it to a latent state of 32 dimensions, a recurrent cell advances the latent given an action (the next phase and a green duration), and a decoder predicts the queues per approach, the pedestrians waiting at each crossing and the departures at the next decision. Multi-step rollout is supported so that the planner can imagine several decisions ahead.

The planner searches over plans of 3 steps. Each step has a phase drawn from a categorical distribution and a green time drawn from a Gaussian that is clipped to 10 to 60 s. At each decision it draws 64 candidate plans, keeps the best 8 and refits the distribution, for 4 iterations. The cost of an imagined rollout adds a weighted sum of predicted queues, a pedestrian-waiting term, a term for waiting minibuses and a steep penalty on the estimated wait of any approach beyond the maximum wait set by the subgoal. The minibus term is approximated from the world model's queue prediction and the currently observed minibus share of each queue, because the world model does not predict vehicle classes. The prototype was run only on a general-purpose CPU.

3.1.2 Slow loop: reasoning that produces a subgoal and a justification

The slow loop is designed to turn what a camera and a clock show into a statement of priorities. Its output is written as reasoning text followed by a JSON subgoal. The reasoning is kept as the justification that goes to the audit log. In the prototype the keyframe is a rendered top-down image of the simulated junction, which stands in for a camera frame. The VLAM backend is configurable, and the open-weight Qwen2.5-VL family (Bai et al., 2025) is used, the 3B model on a CPU for a smoke test and the 7B model planned for a GPU run. Decoding is greedy so that a call can be reproduced.

3.2 The subgoal interface

The subgoal is a compact vector with queue weights for each approach, a pedestrian weight, a maximum wait, a transit priority and a validity period. The permitted ranges are 0.2 to 3.0 for each queue weight, 0.0 to 3.0 for the pedestrian weight, 45 to 180 s for the maximum wait, 0.0 to 2.0 for transit priority and 60 to 900 s for validity. A validator clamps any value outside its range, keeps the previous value when a field is missing or not numeric, and keeps the whole previous subgoal when the reply cannot be parsed. The default subgoal has all weights at 1.0, a maximum wait of 120 s and a validity of 300 s.

This interface has three properties that the design relies on. It is narrow, so it can be sent over a poor link, including a short message. It is bounded, so that a wrong or adversarial value cannot move the cost function outside a fixed region. And it is not a phase command, so even a total failure of the slow loop cannot by itself switch a signal. The one exception is an emergency-vehicle override, which is described but not built and would still pass through the shield.

3.3 The safety shield and the audit

The shield is a rule layer between the planner and the signal controller. Nothing learned can bypass it, and every controller in the evaluation, including the baselines, is routed through it. It enforces a minimum green of 10 s and a maximum green of 90 s, inserts an amber of 3 s and an all-red clearance of 2 s at every phase change, gives every pedestrian phase at least the crossing time (16 s for the east and west crossings and 10 s for the south crossing, from the crossing length at 1.2 m/s plus a 4 s margin), and serves next any approach that has waited longer than 180 s. The design also includes a spillback rule for corridors, which withholds green from a movement whose downstream approach is full and is not exercised at a single junction. Each correction is returned with the rule that caused it and is logged.

The pedestrian minimum is applied to every phase that gives green to a crosswalk, not only when waiting pedestrians are observed, because perception misses some pedestrians. This is stricter than a rule conditioned on detection and is recorded as a deviation from the specification. An independent audit, which reads ground-truth signal and pedestrian state and not the perception output, checks the signal states of every run against the same rules, and it was tested to flag unshielded short greens. The shield is not a proof of road safety, and its parameters have not been checked against Nigerian signal-timing guidance.

3.4 The explainability log

Every decision is written as one JSON line containing the simulated time, controller, scenario and seed, the trigger (scheduled, novelty or none), the connectivity state, the VLAM backend, the reasoning text, the subgoal, the CEM plan, any shield correction with its reason, the applied phase and green, and the world model's one-step error. The log is intended to let a regulator ask why an approach was favoured at a given time and be shown the subgoal, its justification and the triggering conditions. A log records what the model wrote and does not prove that the text caused the weights, so the reasoning is treated as an audit artefact whose reliability must itself be tested, as the smoke test below shows.

3.5 Capability-aware coordinator and degradation tiers

The coordinator is rule-based and not learned. It routes only slow-loop work. In the full design it would send a routine scene to a small edge model, an unusual scene to a large model in the cloud when a link is available, and would treat a rise in the world model's prediction error as a sign that heavier reasoning is needed. An emergency-vehicle alert triggers a direct override.

The prototype implements a subset. It calls the slow loop every 300 simulated seconds and early when a novelty trigger fires. The trigger compares the mean one-step world-model queue error over the last 10 decisions with twice the median of the 30 decisions before them, with at least 120 s between triggers. When the scenario marks the slow loop as unreachable, the coordinator holds the last subgoal, and after more than 600 s offline it switches to a cached time-of-day schedule that was written by hand and not tuned. Each switch is logged. The routing of work between an edge and a cloud model is specified and not implemented.

Table 1**.** Degradation tiers. The prototype implements the hold-last and cached-schedule behaviours and the scenario-defined outage window. The remaining rows are specified, not exercised

Condition	Behaviour
Full connectivity	The slow loop runs a large VLAM in the cloud, the primary tier
Weak connectivity	A small VLM on the edge, or a cached time-of-day subgoal schedule
No connectivity	The fast loop continues on the last subgoal
System fault	A fixed-time signal plan

3.6 Tiered communication

Five channels are planned. Fibre or fourth-generation mobile data carries everything except raw video, short messages over GSM carry alerts and subgoals when data is down, store-and-forward carries logs and training data later, a junction-to-junction mesh over Wi-Fi or LoRa is the main offline path because neighbouring junctions are close, and optional satellite links carry critical alerts only. All incoming messages are intended to be cryptographically signed, and messages may change priorities but can never bypass the shield. This tiering is part of the specified design and is not simulated in the prototype, where connectivity is a binary window in the outage scenario.

3.7 The eight-layer deployment framework

The layer structure used to organise deployment comes from the second author, Dr. Fasina. During the class session of 27 July 2026, they typed the prompt "Design a cloud-based 8-layer framework for AfriCoT-VLA" into DeepSeek on a shared screen, and the layers below are credited to them. The layers are: the sensor layer (cameras and actuators for traffic lights), the network connection (compressed video from cameras over the internet to the cloud), the data store, the model layer (chain-of-thought reasoning, latent world models and the VLAM), the explainability layer, the service layer, the AI agent layer, and an autonomous agentic layer.

There is a tension between this framework and the architecture above. The framework is cloud-based and sends compressed video to the cloud, whereas the architecture keeps raw video on site and runs the fast loop locally. The proposed reconciliation keeps all eight layers, treats the cloud as the full-connectivity tier, and has the network layer carry compressed keyframes for the slow loop and not continuous video. The remaining refinements are the first author's proposals and have not yet been confirmed by the second author. The data store would hold a hot window plus cold history. The model layer would implement the two-timescale design. The service layer would offer decision, query, override and health interfaces. The agent layer would coordinate a corridor, and the autonomous layer would offer plan proposals, what-if simulations and human sign-off. In this reading, any layer above the model layer can fail and the junction falls back to fixed-time control, the network layer is the real bottleneck, and the explainability layer acts as a deployment gate. The framework is not implemented in the prototype beyond the sensing, model and logging functions that the simulation exercises.

3.8 Cloud deployment architecture

This subsection describes the intended deployment and is a design proposal. Nothing in it was built or measured in the cloud: the prototype simulates the control logic and the outage behaviour with a scripted or rule-based backend, and the single 3B model call was made on a local CPU. No latency, cost, availability, security or data-residency figure exists, and none is implied.

Following the second author's framework, the data store, model layer, explainability layer, service layer, agent layer and autonomous layer are intended to be hosted in the cloud, using containers and managed services described generically and without commitment to a vendor. The data store would keep a hot window and cold history, the model layer would host the large VLAM, and the service layer would expose decision, query, override and health interfaces, with the upper layers coordinating corridors and preparing plan proposals for human sign-off. The choice of cloud region is an open design question.

The network layer is the link between site and cloud. It is intended to carry compressed keyframes and context for the slow loop, together with logs and health messages, and not continuous video. Raw video stays on site. Because the slow loop runs every few minutes, a per-call delay of some seconds is tolerable by design. The measured CPU latency of the 3B model is not a cloud figure and should not be read as one.

The edge fast loop and the shield form the resilience tier. They run on site, are never routed to the cloud, and keep the junction operating when the link fails. Each edge node is intended to authenticate to the cloud, each message is intended to be cryptographically signed, and messages may change priorities but never bypass the shield. Cloud components would follow least privilege.

Data residency is an open question for the supervisor and for the regulators, the Lagos State Traffic Management Authority and the Ministry of Transportation. It concerns where keyframes and audit logs may be stored and for how long.

The degradation tiers map onto the role of the cloud as follows. With full connectivity the cloud hosts the large VLAM and all upper layers. With weak connectivity the cloud role shrinks to signed subgoal delivery and log upload, while a small edge model or the cached schedule supplies subgoals. With no connectivity the cloud is absent, the fast loop continues on the last subgoal and logs are held for later upload. On a system fault the signal falls back to the fixed-time plan whatever the cloud is doing.

4 Experimental Setup

4.1 The simulated junction

The prototype uses the SUMO traffic simulator (Alvarez Lopez et al., 2018) with traffic driving on the right, as in Nigeria. The network is one signalised T-junction with an east and west main road of two lanes each way, a south side road of one lane each way, approaches of 300 m, speed limits of 50 km/h on the main road and 40 km/h on the side road, and pedestrian crossings with sidewalks on all three arms. There are three signal phases. Phase A

serves the main road through and right-turn movements together with the permissive east-to-south left turn, with pedestrians crossing the south arm. Phase B is a protected east-to-south left turn with the east-to-west through movement. Phase C serves the side road, with pedestrians crossing the east and west arms. Every change passes through an amber of 3 s and an all-red of 2 s, and a test confirms that no conflicting movements are ever green together.

Four vehicle types are simulated: cars, minibuses (danfos, which count as transit), tricycles (kekes) and motorcycles (okadas). The sublane model lets motorcycles filter between lanes. In the tuning runs the lateral position of okadas had a standard deviation of 0.79 to 0.90 m, against 0.25 to 0.31 m for cars, which is the evidence used here that lane-splitting is represented.

4.2 Scenarios and demand

Each run simulates one hour of measurement after a warm-up of 300 s that is excluded from every metric. Four scenarios are used and are given in the table below. The normal scenario has 700 vehicles per hour per main-road direction and 300 vehicles per hour on the side road. The market scenario raises pedestrian demand and shifts the vehicle mix towards tricycles and motorcycles. The incident scenario closes the kerb lane of the east approach to all demand vehicles from 1200 s to 2100 s of the simulation clock. The outage scenario has market demand and makes the slow loop unreachable from 1200 s to 2400 s.

Table 2**.** Scenario definitions. Flows are vehicles per hour per source, the mix is the share of cars, minibuses, tricycles and motorcycles, and pedestrian demand is in pedestrians per hour in total

Scenario	West, East (veh/h)	South (veh/h)	Mix car/danfo/keke/ okada (%)	Pedestrians (per h)	Event
normal	700, 700	300	45/20/15/20	300	none
market	700, 700	200	30/20/25/25	900	none
incident	700, 700	300	45/20/15/20	300	kerb lane of east approach closed 1200 to 2100 s
outage	700, 700	200	30/20/25/25	900	slow loop unreachable 1200 to 2400 s

Market side-road demand is 200 vehicles per hour, not the 550 first specified, and this is a deviation that matters for interpretation. At 550 vehicles per hour the south arm gridlocked under fixed-time control on the tuning seeds, with a mean insertion delay of 1177 s and up to 395 vehicles never inserted. Candidate values from 550 down to 200 were tried on tuning seeds only, and only 200 passed the criterion of at most five uninserted south vehicles on every seed and a mean south insertion delay below 60 s (at 225 vehicles per hour, up to 28 remained uninserted). The side road therefore carries less traffic in the market scenario than in the normal one. The incident scenario raises waiting only modestly under fixed-time control because one lane of the east approach still carries flow. Seeds 100 to 104 were used for tuning, 200 to 219 for world-model data, and seeds 0 to 4 are the only seeds used for reported evaluation.

4.3 Perception and metrics

Perception in the simulation is derived from the simulator and then degraded: Gaussian noise on counts with a standard deviation of 10.0 percent of the value (minimum 0.5), and each vehicle missed with a probability of 5.0 percent. Controllers see only this noisy output. Per-movement queues are also derived from each halting vehicle's route, with the same noise, because a controller such as max-pressure needs to tell the through and left-turn phases apart. No detector is trained, so the paper says nothing about detection accuracy on real footage.

The metrics are average waiting time (AWT), average travel time (ATT), average queue length (AQL, the mean number of halting vehicles over the incoming lanes), throughput (vehicles arrived) and average pedestrian waiting time. AWT and ATT include insertion delay and cover every vehicle scheduled to depart after the warm-up. Vehicles still in the network or never inserted at the end of the run are right-censored at 3900 s, which makes AWT a lower bound for those vehicles under the same rule for every controller. The rule was introduced after a review found that excluding insertion delay rewarded a controller that starved the side road. Jam teleports are counted as a check on gridlock, and AWT is the primary metric.

4.4 Controllers

Three baselines are built. Fixed-time uses greens of 35 s, 12 s and 25 s for phases A, B and C. The camera-count-plus-timer controller (count-timer) starts each green at 10 s and extends it in 5 s steps while the served queue is at least 3, up to 60 s, cycling A, B, C. Max-pressure picks the phase whose served movements have the largest total queue, with a fixed green of 15 s, and treats the downstream as empty because the junction is isolated. This single-junction form lacks the downstream term that gives max-pressure its network properties.

Two ablations of the fast loop are built. The first, `africot_noslow`, runs the world model and CEM planner with the default subgoal at all times, which is the ablation without the slow loop. The second, `africot_nowm`, replaces the world model by a one-step queue-discharge heuristic that uses smoothed arrival rates and fixed discharge rates, which are untuned engineering estimates. A third variant, `africot_full`, adds the slow loop. No planner parameter was tuned on any seed.

The world model was trained on windows from fixed-time, count-timer, max-pressure and random-timing controllers in the normal, market and incident scenarios over 25 data seeds, with 32906 training, 11027 validation and 11116 test windows split by seed. The outage scenario was skipped because, for controllers without a slow loop, it would duplicate market episodes. Training used the Adam optimiser with early stopping on the validation split, and the test seeds are 215 to 219.

Table 3**.** Specified compared with built. A component is built if it exists in the prototype and has been run, and wiring-tested if it was run only with a rule-based stand-in for the model

Component	Status
Edge perception with noise	Built, simulated from SUMO state, no detector
Latent world model and CEM planner	Built and evaluated
Safety shield and independent audit	Built and evaluated

Component	Status
Explainability log	Built, one line per decision
Subgoal validator	Built and tested
Coordinator (schedule, novelty, hold-last, cached schedule)	Built, wiring-tested
VLAM slow loop (africot_full)	Implemented, wiring-tested with a mock backend, one 3B smoke test, not yet evaluated
Capability-aware routing between edge and cloud models	Specified, not built
Emergency-vehicle override	Specified, not built
Tiered communication and signed messages	Specified, not built
Eight-layer service, agent and autonomous layers	Specified, not built
Cloud hosting of layers 3 to 8, keyframe transport, edge authentication	Specified, not built, nothing measured in the cloud
Deep reinforcement-learning baseline	Not built
LLMLight baseline	Not built
Multi-junction corridor and neighbour exchange	Specified, not built

4.5 Reproducibility and statistics

Every run writes a result file with its metrics, configuration, backend and code revision, and the reported tables are regenerated from these files by script. The test suite of 68 tests passed in a clean reproduction, and re-running the five built controllers on one scenario and seed gave metrics identical to the committed files. Means and standard deviations are over seeds 0 to 4. Because five seeds give little power, a per-seed paired comparison is added for selected pairs, reporting the mean difference, the number of seeds in which the first controller is lower, a paired t-test and an exact Wilcoxon signed-rank test. With five pairs the smallest possible exact two-sided Wilcoxon p-value is 0.0625, and no correction for multiple comparisons is applied, so these tests are exploratory and a gap that lies within one standard deviation is not described as a difference.

5 Results

5.1 The world model against persistence

The world model was checked on seeds that were never used for training or validation. The figure below compares its error with that of the persistence guess that queues stay unchanged, which is the natural reference for a planner that needs to predict how queues will change.

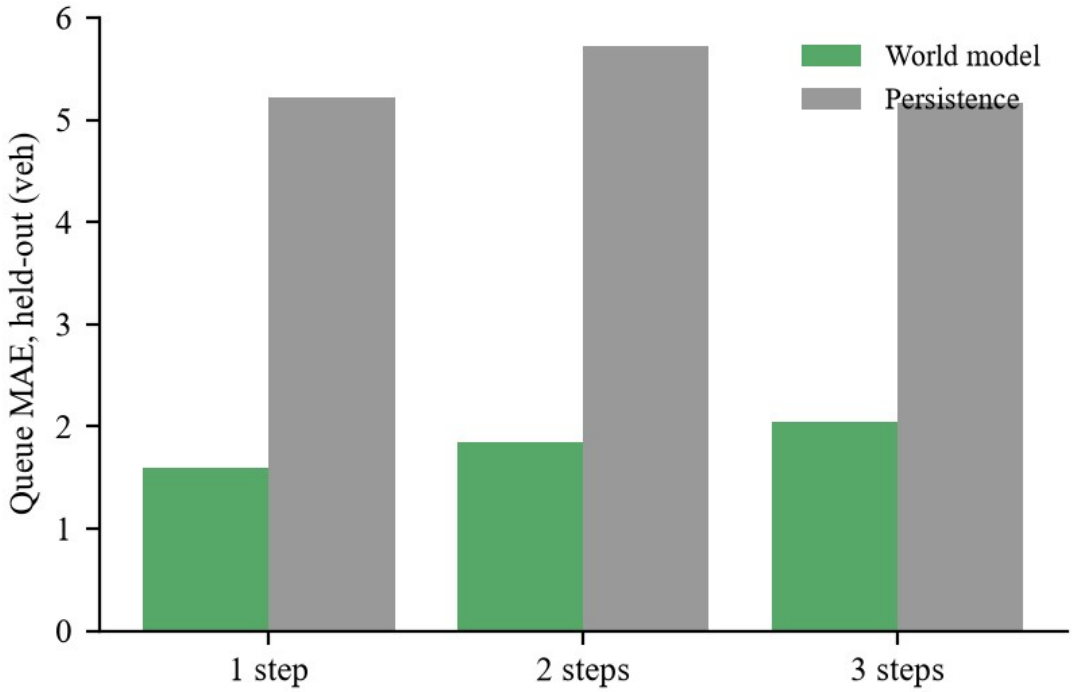


Figure 2. Held-out mean absolute error of the latent world model and of the persistence guess for one-, two- and three-step predictions

At one step ahead the mean absolute queue error was 1.60 vehicles for the model and 5.23 for persistence, an improvement of 69.4 percent, and the advantage remained at three steps (2.05 against 5.16, 60.3 percent). The specified check, a one-step improvement of at least 10 percent, was passed by a wide margin. Errors are measured against noisy observations, as the deployed controller would see, so they are not errors against the true queue. Persistence is a weak reference when the signal changes between decisions, so the model's value for planning is tested only through the controller results below.

Table 4. World-model errors on held-out data, with the persistence guess for comparison

Horizon	Queue MAE model	Queue MAE persistence	Improvement (%)	Pedestrian MAE model	Pedestrian MAE persistence	Departure MAE model	Departure MAE persistence
1 step	1.60	5.23	69.4	0.64	1.19	0.88	4.75
2 steps	1.84	5.72	67.8	0.72	1.38	0.96	4.18
3 steps	2.05	5.16	60.3	0.73	1.32	1.01	3.42

Inside the controller runs, the world model's online one-step queue error for `africot_noslow` averaged 0.99 vehicles in the normal scenario, 1.10 in market and 1.50 in incident, which is higher than on the held-out set. One plausible reason is that the planner visits states that differ from those of the data-collecting controllers, which is also why the novelty trigger is based on this error.

5.2 Controller comparison

The next table gives the mean and standard deviation of AWT over the five evaluation seeds for the five controllers in four scenarios, and the figure below shows the same data.

Table 5. Average waiting time (s), mean and standard deviation over seeds 0 to 4

Scenario	fixed_time	count_timer	max_pressure	africot_nowm	africot_noslow
normal	36.3 (10.1)	38.1 (1.3)	17.0 (1.1)	39.9 (10.2)	19.3 (1.5)
market	42.4 (11.1)	44.1 (4.3)	25.9 (13.5)	48.2 (9.1)	23.8 (4.4)
incident	56.8 (6.8)	52.0 (7.4)	35.8 (8.6)	72.6 (29.1)	39.9 (9.2)
outage	42.4 (11.1)	44.1 (4.3)	25.9 (13.5)	48.2 (9.1)	23.8 (4.4)

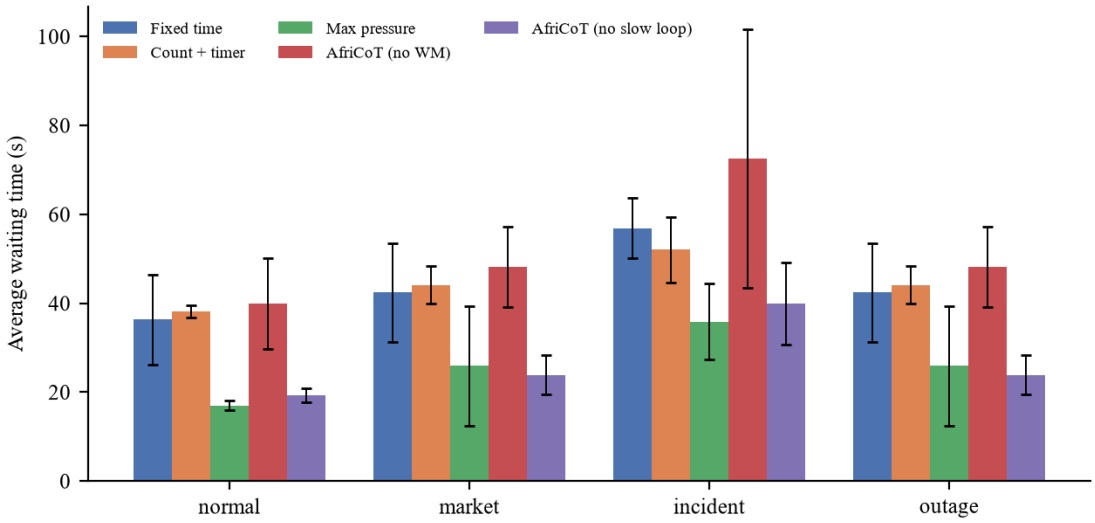


Figure 3. Average waiting time by scenario and controller, mean over seeds 0 to 4 with standard deviation as error bars

On normal demand, max-pressure had the lowest mean AWT (17.0 s), `africot_noslow` was close behind (19.3 s), and fixed-time (36.3 s), count-timer (38.1 s) and `africot_nowm` (39.9 s) were well behind. In the paired by-seed comparison, `africot_noslow` was higher than max-pressure by 2.3 s on average, and was lower in only 0 of 5 seeds, so max-pressure was lower in every seed. The planner with a default subgoal was lower than fixed-time in every seed (5 of 5, mean difference -17.1 s).

On market demand, max-pressure had a mean AWT of 25.9 s with a standard deviation of 13.5 s and `africot_noslow` had 23.8 s with a standard deviation of 4.4 s. The lead of `africot_noslow` in the means is -2.1 s, well inside one standard deviation, and it was lower than max-pressure in only 1 of 5 seeds, so the two are described as comparable. Both were lower than fixed-time (42.4 s) in every seed.

On incident demand, the mean AWT was 35.8 s for max-pressure and 39.9 s for `africot_noslow`. The planner with a default subgoal was higher in the means, was lower than max-pressure in only 1 of 5 seeds, and the mean difference of 4.1 s is smaller than the spread between seeds (15.2 s), so the data do not separate the two with confidence, but they offer no support for an advantage of the planner. Both were lower than fixed-time (56.8 s) in every seed.

The outage rows equal the market rows for every controller without a slow loop, because the outage window changes nothing when no slow loop exists, so outage is not an independent scenario for them and is not tested separately.

Table 6**.** All reported metrics per scenario and controller (mean and standard deviation over seeds 0 to 4)

Scenario	Controller	AWT (s)	ATT (s)	AQL (veh)	Throughput	Ped. wait (s)	Violations
normal	fixed_time	36.3 (10.1)	100.8 (14.3)	16.8 (4.8)	1698 (26)	19.0 (0.9)	0
normal	count_timer	38.1 (1.3)	104.0 (2.1)	17.8 (0.6)	1704 (17)	13.0 (1.2)	0
normal	max_pressure	17.0 (1.1)	76.5 (1.6)	7.6 (0.6)	1715 (21)	12.5 (0.8)	0
normal	africot_nowm	39.9 (10.2)	104.3 (12.5)	18.2 (4.4)	1702 (15)	19.3 (3.7)	0
normal	africot_noslow	19.3 (1.5)	79.9 (2.3)	8.6 (0.9)	1711 (18)	11.6 (1.1)	0
market	fixed_time	42.4 (11.1)	109.4 (15.5)	18.3 (4.8)	1597 (28)	20.3 (0.4)	0
market	count_timer	44.1 (4.3)	111.5 (5.7)	19.2 (2.0)	1601 (20)	14.1 (0.3)	0
market	max_pressure	25.9 (13.5)	88.7 (18.0)	10.7 (5.6)	1598 (31)	15.5 (5.4)	0
market	africot_nowm	48.2 (9.1)	115.9 (12.0)	20.8 (4.2)	1590 (27)	19.4 (1.6)	0
market	africot_noslow	23.8 (4.4)	86.2 (6.5)	9.8 (1.9)	1601 (24)	12.0 (0.8)	0
incident	fixed_time	56.8 (6.8)	132.9 (9.7)	23.0 (3.9)	1697 (20)	18.9 (0.9)	0
incident	count_timer	52.0 (7.4)	125.6 (10.0)	23.7 (2.7)	1702 (14)	16.5 (0.8)	0
incident	max_pressure	35.8 (8.6)	108.3 (10.9)	15.7 (4.1)	1713 (23)	13.8 (1.0)	0
incident	africot_nowm	72.6 (29.1)	147.5 (33.1)	28.5 (6.1)	1690 (10)	22.2 (3.6)	0
incident	africot_noslow	39.9 (9.2)	115.3 (14.1)	16.9 (2.9)	1708 (18)	14.0 (0.9)	0
outage	fixed_time	42.4 (11.1)	109.4 (15.5)	18.3 (4.8)	1597 (28)	20.3 (0.4)	0
outage	count_timer	44.1 (4.3)	111.5 (5.7)	19.2 (2.0)	1601 (20)	14.1 (0.3)	0
outage	max_pressure	25.9 (13.5)	88.7 (18.0)	10.7 (5.6)	1598 (31)	15.5 (5.4)	0
outage	africot_nowm	48.2 (9.1)	115.9 (12.0)	20.8 (4.2)	1590 (27)	19.4 (1.6)	0
outage	africot_noslow	23.8 (4.4)	86.2 (6.5)	9.8 (1.9)	1601 (24)	12.0 (0.8)	0

Throughput barely separates the controllers: in the normal scenario it ranged from 1698 to 1715 vehicles per run across controllers, with standard deviations of up to 26, so AWT, ATT and AQL carry the information. The average pedestrian waiting time was lower for two of the adaptive controllers than for fixed-time on normal demand (11.6 s for `africot_noslow` and 12.5 s for max-pressure, against 19.0 s), although the shield's pedestrian minimum applies to every controller.

Table 7**.** Exploratory paired by-seed comparisons of AWT over seeds 0 to 4 (n = 5), without correction for multiple comparisons

Scenario	Comparison (A versus B)	Mean AWT difference, A minus B (s)	SD of difference (s)	Paired t-test p	Wilcoxon p	Seeds with A lower
normal	africot_noslow vs max_pressure	2.3	1.1	0.011	0.0625	0 of 5
normal	africot_noslow vs fixed_time	-17.1	9.2	0.015	0.0625	5 of 5
normal	max_pressure vs fixed_time	-19.4	10.1	0.013	0.0625	5 of 5
normal	africot_nowm vs fixed_time	3.6	16.4	0.652	0.4375	1 of 5
normal	africot_noslow vs africot_nowm	-20.6	10.8	0.013	0.0625	5 of 5
market	africot_noslow vs max_pressure	-2.1	15.7	0.780	0.6250	1 of 5
market	africot_noslow vs fixed_time	-18.6	13.4	0.036	0.0625	5 of 5
market	max_pressure vs fixed_time	-16.5	8.0	0.010	0.0625	5 of 5
market	africot_nowm	5.8	17.3	0.498	0.8125	2 of 5

Scenario	Comparison (A versus B)	Mean AWT difference, A minus B (s)	SD of difference (s)	Paired t-test p	Wilcoxon p	Seeds with A lower
market	africot_noslow vs fixed_time	-24.3	10.6	0.007	0.0625	5 of 5
incident	africot_noslow vs africot_nowm	4.1	15.2	0.578	0.4375	1 of 5
incident	africot_noslow vs max_pressure	-16.9	12.2	0.036	0.0625	5 of 5
incident	africot_noslow vs fixed_time	-21.0	3.7	0.000	0.0625	5 of 5
incident	max_pressure vs fixed_time	15.7	30.8	0.317	0.4375	1 of 5
incident	africot_nowm vs fixed_time	-32.6	23.7	0.037	0.0625	5 of 5

5.3 Ablations and negative results

Removing the world model did not produce a competitive controller. The `africot_nowm` variant was worse than fixed-time in the means on all three scenarios (normal by 3.6 s, market by 5.8 s and incident by 15.7 s), although none of these differences was consistent across seeds (it was lower than fixed-time in 1, 2 and 1 of 5 seeds). The variant with the world model was lower than the one without it in every seed in every scenario tested (mean difference -20.6 s on normal). The discharge rates of the heuristic are untuned guesses, so this is an ablation of one specific heuristic and not a bound on what a world-model-free planner can reach, and the heuristic variant also produced the only jam teleport among the planner runs (1, on normal demand) as well as vehicles that were never inserted in some runs.

The planner with a default subgoal did not beat the strongest classical baseline. No deeper investigation was carried out. Candidate explanations that are untested include the cost being summed per plan step and not weighted by the duration of each step, the short horizon, the approximation of the minibus term, the noisy observations, and the use of an untuned default subgoal. All comparisons are between shielded controllers.

Further observations temper the tables. Count-timer was not better than fixed-time on normal or market demand in the means, and the gap in each case lies within one standard deviation. An earlier version of the metrics, which excluded insertion delay, ranked the adaptive baselines differently from fixed-time on market demand, and the ranking above is the one that remains after the correction described in the setup. Max-pressure shows a large spread on market demand (13.5 s), which makes its market result less certain than its normal result. Max-pressure had 1 jam teleport on market demand, with additional non-jam teleports in several cells.

5.4 Shield audit

All 100 evaluation runs passed through the shield, and the independent audit found 0 safety violations. The shield was nonetheless active. Across the runs, it made 4684 pedestrian-crossing corrections, 116 maximum-green corrections and 2 starvation corrections.

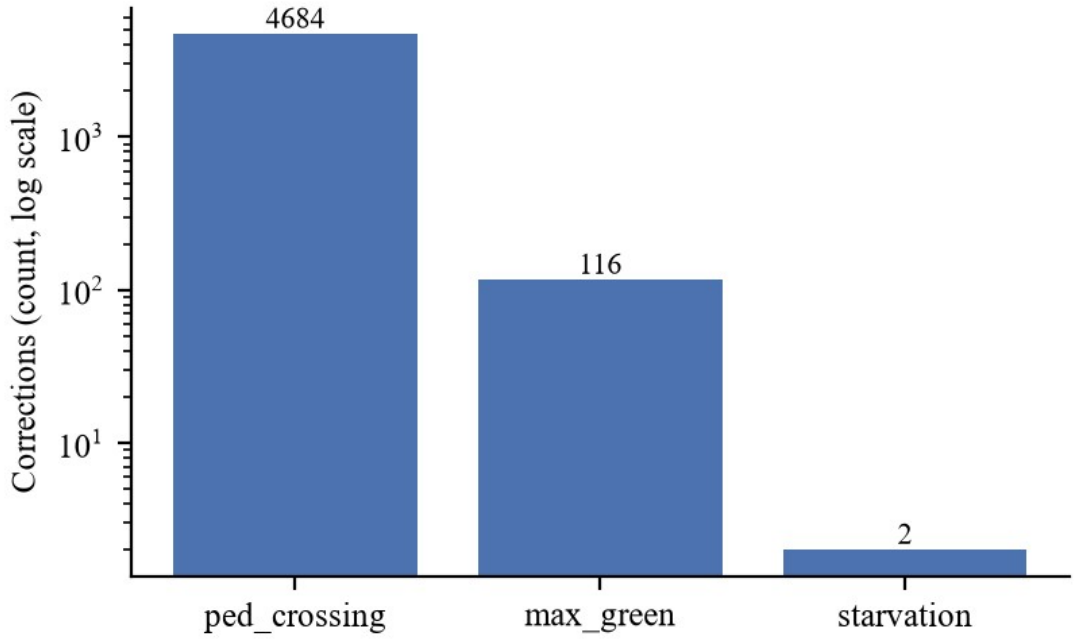


Figure 4. Shield corrections by controller and rule, summed over all evaluation runs

Table 8. Shield corrections by controller and rule, summed over the evaluation runs

Controller	Pedestrian crossing	Maximum green	Starvation guard	Total corrections	Safety violations
count_timer	1328	0	0	1328	0
max_pressure	1505	4	2	1511	0
africot_nowm	433	112	0	545	0
africot_noslow	1418	0	0	1418	0

Most corrections were pedestrian-crossing minima, which lift side-road greens that were shorter than the crossing time, and which were issued to max-pressure (1505), `africot_noslow` (1418) and count-timer (1328). The `africot_nowm` variant received 112 maximum-green corrections, more than any other, and max-pressure received 4 and 2 starvation corrections. Fixed-time received none. The counts show that the planners propose timings that the rules reject and that the shield changes what is applied. They do not show that the rules themselves are the right ones.

5.5 Slow-loop wiring and the 3B smoke test

The integrated controller was run with a mock backend on all four scenarios and five seeds, giving 20 runs. The mock backend is deterministic and rule-based, it is not a vision-language model, and its outputs are not VLAM results. The purpose was to check the wiring. All runs completed, with 0 safety violations and 0 parse failures, and with 9 to 15 slow-loop calls per run.

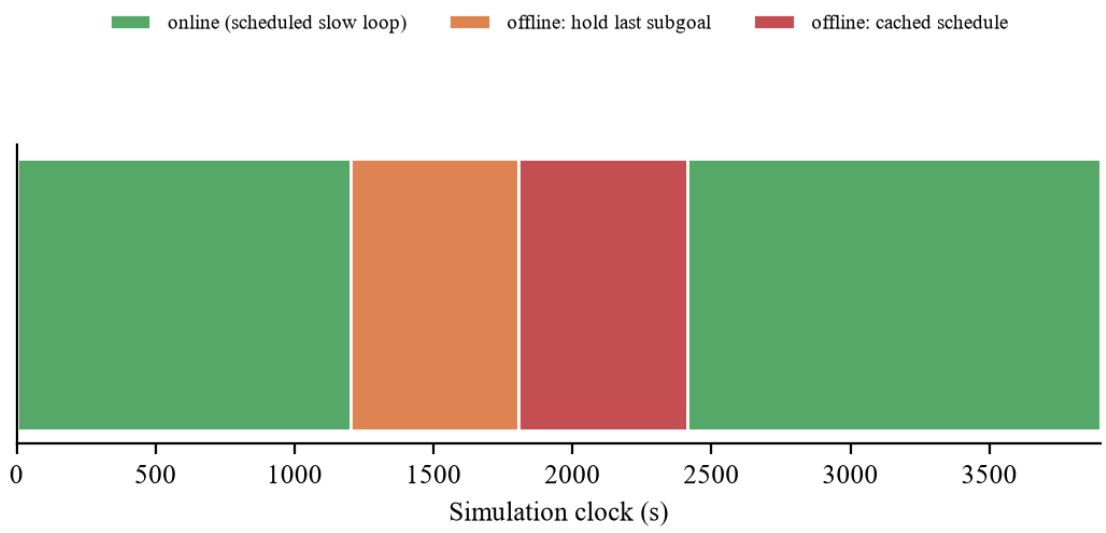


Figure 5. Connectivity states of one integration run in the outage scenario with the mock backend (seed 0). The timeline shows scripted behaviour and is not a VLAM result

In the outage scenario, the log shows the intended sequence on every seed. The slow loop was online until 1200 s, the last subgoal was held, the cached schedule was used from about 1807 s, and connectivity returned at about 2413 s. The novelty trigger fired inside the incident window on all five seeds, but it is not specific to incidents. It also fired in the normal scenario on seeds 3 and 4, in the market scenario on seeds 1, 3 and 4, and in the outage scenario on seed 4. The trigger therefore indicates that the world model's error rises at times, and not that an incident has occurred.

A single call to Qwen2.5-VL-3B (Bai et al., 2025) was made on a rendered market-day snapshot on a four-core CPU with greedy decoding. The reply parsed and validated, with a pedestrian weight of 3.0 and all other values at their defaults, and the call took 146.7 s on an idle machine. The context gave about 10.9 waiting pedestrians at the east crossing and about 2.3 at the west crossing. The model's reasoning attributed the waiting pedestrians to the west arm, so its chain of thought is unreliable at this size. This single call shows that the pipeline from image to validated subgoal runs, and it is no evidence of sound reasoning. It also shows that a logged justification can be wrong while the resulting subgoal is plausible. A re-run on a separate download gave identical text, consistent with deterministic decoding.

No evaluation of the full controller with a vision-language model is reported in this version. The results of the planned GPU grid with the 7B model are to be added here, in the same scenarios and seeds as the table above.

6 Discussion

The most secure findings concern the parts that were evaluated. The small world model predicts queue changes at the next few decisions much better than a persistence guess, the planner built on it is lower than fixed-time control in every seed in the normal, market and incident scenarios and lower than the same planner with a crude heuristic in place of the world model, and the shield produced no violations while correcting thousands of proposals. Taken together, these results support the narrower claim that a learned world model can be

used inside a shielded planner at a single junction without the safety record being damaged by the learned parts. They do not support the stronger claim that this planner is the best available controller, because max-pressure is lower in the means on normal and incident demand and comparable on market demand.

That result does not dismiss the architecture, but it repositions the question. If the fast loop alone is not ahead of a simple actuated rule at one isolated junction, the case for the architecture cannot rest on average delay in ordinary conditions. It would have to rest on situations in which a rule that sees only queues is blind, such as an event that changes priorities in a way queue counts do not show, or the need for an audit trail. The prototype's scenarios contain such situations only weakly, and the market scenario was made less demanding on the side road to avoid gridlock, so they are a limited test of what the slow loop is meant to address.

The working hypothesis for the slow loop is that context a camera and a calendar supply, such as a market day or a blocked lane, could be translated into subgoal weights that the fast loop alone cannot infer, and that the written justification could make such changes reviewable. This hypothesis has not been tested. It would be supported only if the full controller shows lower waiting than the planner with a default subgoal in paired by-seed comparisons, with the log showing sensible reasoning, and it would be weakened if the reasoning of the larger model were also unreliable.

The smoke test carries a methodological lesson whatever the later experiment finds. The 3B model's reasoning was fluent and wrong about where pedestrians were waiting, and a log that stores such text would mislead an auditor who took it at face value. The design therefore asks for two further safeguards that are not yet built: comparing the claims in the reasoning with the perception output that was available to the model, and flagging decisions for manual review when the two disagree. The shield and the subgoal bounds limit the harm that a wrong rationale can cause, but they do not make the rationale correct.

The degradation tiers work as wired, but the prototype does not show how much control quality is lost when they are used, because the controller they would protect has not been evaluated. The ablations under connection loss, with and without neighbour information, and with fixed and capability-aware routing remain to be run.

Finally, the cloud orientation of the eight-layer framework sits uneasily with sites where connectivity is the main constraint. Hosting the upper layers in the cloud while keeping the fast loop and shield on site preserves the layer structure and the resilience of the junction, and it is a proposal for confirmation by the second author and not a settled design.

7 Limitations

The evidence has several limits that readers should keep in mind.

- A single simulated junction is used. The corridor and neighbour exchange that the design describes for close junctions are not built, so claims about coordination and green waves are outside the evidence.

- The evaluation is in simulation with perception derived from simulator state plus noise. No detector was trained or tested on real footage, and the renderer used as the slow-loop keyframe is a simulator view and not a camera image.
- Five evaluation seeds give low power. The paired tests are exploratory and uncorrected, and many cells have a large standard deviation.
- The full system has not been evaluated. The only real-model evidence is one call of a 3B model, and the integration runs used a rule-based mock backend. The slow loop is treated as instantaneous in simulated time, although the measured call time of the 3B model on an idle four-core CPU was 147 s and a larger model on the same hardware would take longer. The validity period in the subgoal is validated but not used by the coordinator.
- The deep reinforcement-learning and LLMLight baselines are not built, so the paper makes no comparison with learned traffic controllers or language-model controllers. The max-pressure baseline is its isolated single-junction form.
- Market side-road demand was lowered from 550 to 200 vehicles per hour to avoid gridlock, which changes the character of that scenario. The incident scenario is mild under fixed-time control, and the outage scenario duplicates market for every controller without a slow loop.
- The novelty trigger fires without an incident, and the cached schedule, the heuristic discharge rates and the minibus cost term are untuned or approximate.
- No cloud deployment was built. Latency, availability, cost, security and data residency of the cloud-hosted layers are unmeasured, and the connectivity model is a binary outage window.
- The communication tiers, capability-aware routing, emergency override and the upper framework layers are specified and not built. The explainability log was not assessed with regulators.
- The hardware roadmap (hardware-in-the-loop testing, shadow mode, supervised pilot) is described and not carried out.

8 Conclusion

This paper proposed AfriCoT-VLA, a cloud-hosted, two-timescale traffic signal design for infrastructure-constrained cities, in which a chain-of-thought vision-language slow loop in the cloud sets a bounded subgoal and a justification, an on-site latent world model and CEM planner choose timings against it, a rule-based shield limits what any learned component can do, the on-site loop and shield form a resilience tier for link failures, and a capability-aware coordinator and tiered communication govern degradation. Relative to the authors' earlier position paper, the contribution is the reasoning slow loop with its justification log, the shield and audit, the coordinator and degradation tiers, and simulation evidence that includes negative results.

In one simulated junction and five seeds, the world model predicted short-horizon queues much better than persistence and the shield produced no violations, but the planner with a default subgoal was not better than max-pressure control, and removing the world model gave a controller no better than fixed-time control. The evidence supports the use of the fast

loop inside a shield as a basis for further work, and it does not support a claim that the system improves on simple actuated control.

The full controller, africot_full, is implemented and wiring-tested with a mock backend and has had one smoke test with a 3B model, but it has not been evaluated, so no conclusion about the benefit of the slow loop is drawn. Its evaluation with a 7B model on a GPU, in the same scenarios and seeds, is the next step.

Further work comprises cloud deployment and its measurement (latency, availability, cost, security testing and the data-residency question), staged through hardware-in-the-loop testing, shadow mode and a supervised pilot, together with the missing deep reinforcement-learning and LLMLight baselines, a corridor of two to three junctions with neighbour information, tests of the reasoning against the perception it was given, and a supervised pilot with override before live operation.

References

Adeyemo, O. (2026, June 9). Could AI help Lagos untangle its never-ending traffic jam? Business A.M. https://businessamlive.com/could-ai-help-lagos-untangle-its-never-ending-traffic-jam/

Alvarez Lopez, P., Wiessner, E., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., & Wagner, P. (2018). Microscopic traffic simulation using SUMO. In 2018 21st International Conference on Intelligent Transportation Systems (ITSC) (pp. 2575–2582). IEEE. https://doi.org/10.1109/ITSC.2018.8569938

Ariyo, T. W., & Fasina, E. P. (2026). Towards a unified framework of vision-language-action models and latent world models for low-resource, infrastructure-constrained environments in Sub-Saharan Africa [Manuscript submitted for publication]. Machine Learning Research Group, Department of Artificial Intelligence and Robotics, University of Lagos.

Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., Gejji, A., Martin, A., Hogan, F. R., Dugas, D., Bojanowski, P., Khalidov, V., Labatut, P., . . . Ballas, N. (2025). V-JEPA 2: Self-supervised video models enable understanding, prediction and planning (arXiv:2506.09985). arXiv. https://doi.org/10.48550/arXiv.2506.09985

Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., Zhong, H., Zhu, Y., Yang, M., Li, Z., Wan, J., Wang, P., Ding, W., Fu, Z., Xu, Y., . . . Lin, J. (2025). Qwen2.5-VL technical report (arXiv:2502.13923). arXiv. https://doi.org/10.48550/arXiv.2502.13923

Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., . . . Zhilinsky, U. (2025). π0: A vision-language-action flow model for general robot control. In Robotics: Science and Systems XXI. https://doi.org/10.15607/RSS.2025.XXI.010

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J. Q., Demszky, D., . . . Liang,

P. (2021). On the opportunities and risks of foundation models (arXiv:2108.07258). arXiv. https://doi.org/10.48550/arXiv.2108.07258

Chu, T., Wang, J., Codecà, L., & Li, Z. (2020). Multi-agent deep reinforcement learning for large-scale traffic signal control. IEEE Transactions on Intelligent Transportation Systems, 21(3), 1086–1095. https://doi.org/10.1109/TITS.2019.2901791

Chua, K., Calandra, R., McAllister, R., & Levine, S. (2018). Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In Advances in Neural Information Processing Systems (Vol. 31). Curran Associates. https://doi.org/10.48550/arXiv.1805.12114

de Boer, P.-T., Kroese, D. P., Mannor, S., & Rubinstein, R. Y. (2005). A tutorial on the cross-entropy method. Annals of Operations Research, 134(1), 19–67. https://doi.org/10.1007/s10479-005-5724-z

Feng, T., Wang, X., Jiang, Y.-G., & Zhu, W. (2025). Embodied AI: From LLMs to world models. IEEE Circuits and Systems Magazine, 25(4), 14–37. https://doi.org/10.1109/MCAS.2025.3603693

Figure AI. (2025, February 20). Helix: A vision-language-action model for generalist humanoid control [Technical report]. https://www.figure.ai/news/helix

GSMA. (2024a). The mobile economy Sub-Saharan Africa 2024. GSMA Intelligence. https://www.gsma.com/solutions-and-impact/connectivity-for-good/mobile-economy/wp-content/uploads/2024/11/GSMA_ME_SSA_2024_Web.pdf

GSMA. (2024b). The state of mobile internet connectivity report 2024. https://gsma.com/r/wp-content/uploads/2024/10/The-State-of-Mobile-Internet-Connectivity-Report-2024.pdf

Hafner, D., Lee, K.-H., Fischer, I., & Abbeel, P. (2022). Deep hierarchical planning from pixels. In Advances in Neural Information Processing Systems (Vol. 35). https://doi.org/10.48550/arXiv.2206.04114

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning latent dynamics for planning from pixels. In Proceedings of the 36th International Conference on Machine Learning (Vol. 97, pp. 2555–2565). PMLR. https://doi.org/10.48550/arXiv.1811.04551

Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Mastering diverse control tasks through world models. Nature, 640(8059), 647–653. https://doi.org/10.1038/s41586-025-08744-2

Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, robust world models for continuous control. In The Twelfth International Conference on Learning Representations (ICLR 2024). https://doi.org/10.48550/arXiv.2310.16828

International Energy Agency, International Renewable Energy Agency, United Nations Statistics Division, World Bank, & World Health Organization. (2025). Tracking SDG 7: The energy progress report 2025. World Bank. https://www.iea.org/reports/tracking-sdg7-the-energy-progress-report-2025

ITEdgeNews. (2026, April 29). LAMATA deploys AI-powered smart traffic systems to cut congestion on Lagos roads. https://www.itedgenews.africa/lamata-deploys-ai-powered-smart-traffic-systems-to-cut-congestion-on-lagos-roads/

Kenya Urban Roads Authority. (2023, February 28). Intelligent transport systems. https://kura.go.ke/news/intelligent-transport-systems/13363/

Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E. P., Sanketi, P. R., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., & Finn, C. (2024). OpenVLA: An open-source vision-language-action model. In Proceedings of the 8th Conference on Robot Learning (Vol. 270, pp. 2679–2713). PMLR. https://doi.org/10.48550/arXiv.2406.09246

Kinyanjui, M. (2024, November 27). Phase one of Sh7.18bn Nairobi Intelligent Traffic System to kick off in March. The Eastleigh Voice. https://eastleighvoice.co.ke/city-affairs/92971/phase-one-of-sh7.18bn-nairobi-intelligent-traffic-system-to-kick-off-in-march

Lai, S., Xu, Z., Zhang, W., Liu, H., & Xiong, H. (2023). LLMLight: Large language models as traffic signal control agents (arXiv:2312.16044). arXiv. https://doi.org/10.48550/arXiv.2312.16044

Liu, S., Tang, J., Hassan, M. S., Sharkawy, M. H., Fouda, M. M. G., Shang, T., & Wang, Z. (2026). Infrastructure first: Enabling embodied AI for science in the Global South (arXiv:2604.06722). arXiv. https://doi.org/10.48550/arXiv.2604.06722

Mumbi, L. (2026, May 12). Government to roll out AI-powered traffic lights at 125 Nairobi junctions. The Eastleigh Voice. https://eastleighvoice.co.ke/city-affairs/348540/government-to-roll-out-ai-powered-traffic-lights-at-125-nairobi-junctions

NVIDIA, Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., Jang, J., Jiang, Z., Kautz, J., Kundalia, K., Lao, L., Li, Z., Lin, Z., Lin, K., . . . Zhu, Y. (2025). GR00T N1: An open foundation model for generalist humanoid robots (arXiv:2503.14734). arXiv. https://doi.org/10.48550/arXiv.2503.14734

Octo Model Team, Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., Luo, J., Tan, Y. L., Chen, L. Y., Sanketi, P., Vuong, Q., Xiao, T., Sadigh, D., Finn, C., & Levine, S. (2024). Octo: An open-source generalist robot policy. In Robotics: Science and Systems XX. https://doi.org/10.15607/RSS.2024.XX.090

Olomu, S. (2025, February 7). Lagos picks Huawei to deploy intelligent traffic system. ITWeb Africa. https://itweb.africa/content/JBwEr7n3ngKM6Db2

Rubinstein, R. (1999). The cross-entropy method for combinatorial and continuous optimization. Methodology and Computing in Applied Probability, 1(2), 127–190. https://doi.org/10.1023/A:1010091220143

Sapkota, R., Cao, Y., Roumeliotis, K. I., & Karkee, M. (2025). Vision-language-action (VLA) models: Concepts, progress, applications and challenges (arXiv:2505.04769). arXiv. https://doi.org/10.48550/arXiv.2505.04769

Shi, L. X., Ichter, B., Equi, M., Ke, L., Pertsch, K., Vuong, Q., Tanner, J., Walling, A., Wang, H., Fusai, N., Li-Bell, A., Driess, D., Groom, L., Levine, S., & Finn, C. (2025). Hi Robot: Open-ended instruction following with hierarchical vision-language-action models. In Proceedings of the 42nd International Conference on Machine Learning (Vol. 267). PMLR. https://doi.org/10.48550/arXiv.2502.19417

Varaiya, P. (2013). Max pressure control of a network of signalized intersections. Transportation Research Part C: Emerging Technologies, 36, 177–195. https://doi.org/10.1016/j.trc.2013.08.014

Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., & Li, Z. (2019a). PressLight: Learning max pressure control to coordinate traffic signals in arterial network. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 1290–1298). ACM. https://doi.org/10.1145/3292500.3330949

Wei, H., Xu, N., Zhang, H., Zheng, G., Zang, X., Chen, C., Zhang, W., Zhu, Y., Xu, K., & Li, Z. (2019b). CoLight: Learning network-level cooperation for traffic signal control. In Proceedings of the 28th ACM International Conference on Information and Knowledge Management (pp. 1913–1922). ACM. https://doi.org/10.1145/3357384.3357902

Wei, H., Zheng, G., Gayah, V., & Li, Z. (2019c). A survey on traffic signal control methods (arXiv:1904.08117). arXiv. https://doi.org/10.48550/arXiv.1904.08117

Yang, S., Li, H., Wang, B., Chen, Y., Tian, Y., Wang, T., Wang, H., Zhao, F., Liao, Y., & Pang, J. (2025). InstructVLA: Vision-language-action instruction tuning from understanding to manipulation (arXiv:2507.17520). arXiv. https://doi.org/10.48550/arXiv.2507.17520

Yin, S., Pang, X., Ding, Y., Chen, M., Bi, Y., Xiong, Y., Huang, W., Xiang, Z., Shao, J., & Chen, S. (2024). SafeAgentBench: A benchmark for safe task planning of embodied LLM agents (arXiv:2412.13178). arXiv. https://doi.org/10.48550/arXiv.2412.13178

Zawalski, M., Chen, W., Pertsch, K., Mees, O., Finn, C., & Levine, S. (2024). Robotic control via embodied chain-of-thought reasoning. In Proceedings of the 8th Conference on Robot Learning (Vol. 270, pp. 3157–3181). PMLR. https://doi.org/10.48550/arXiv.2407.08693

Zhang, W., Kong, X., Bräunl, T., & Hong, J. B. (2024). SafeEmbodAI: A safety framework for mobile robots in embodied AI systems (arXiv:2409.01630). arXiv. https://doi.org/10.48550/arXiv.2409.01630

Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., Vuong, Q., Vanhoucke, V., Tran, H., Soricut, R., Singh, A., Singh, J., Sermanet, P., Sanketi, P. R., Salazar, G., . . . Han, K. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. In Proceedings of the 7th Conference on Robot Learning (Vol. 229, pp. 2165–2183). PMLR. https://doi.org/10.48550/arXiv.2307.15818